Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
GRADUATION THESIS

Разработка модуля управления товарными запасами и продажами для ERP-системы / Development of Inventory and Sales Module for ERP System

Обучающийся / Student Джахан Исрат
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет информационных технологий и программирования
Группа/Group M34091
Направление подготовки/ Subject area 09.03.02 Информационные системы и технологии
Образовательная программа / Educational program Программирование и интернет-технологии 2021
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Руководитель ВКР/ Thesis supervisor Андреев Николай Владимирович, Университет ИТМО, факультет информационных технологий и программирования, заведующий лабораторией

Обучающийся/Student

Документ подписан
Джахан Исрат
20.05.2025

(эл. подпись/ signature)

Джахан Исрат
(Фамилия И.О./ name and surname)

Руководитель ВКР/ Thesis supervisor

Документ подписан
Андреев Николай Владимирович
20.05.2025

(эл. подпись/ signature)

Андреев Николай Владимирович
(Фамилия И.О./ name and surname)

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ / OBJECTIVES FOR A GRADUATION THESIS

Обучающийся / Student Джахан Исрат
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет информационных технологий и программирования
Группа/Group M34091
Направление подготовки/ Subject area 09.03.02 Информационные системы и технологии
Образовательная программа / Educational program Программирование и интернет-технологии 2021
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Тема ВКР/ Thesis topic Разработка модуля управления товарными запасами и продажами для ERP-системы / Development of Inventory and Sales Module for ERP System
Руководитель ВКР/ Thesis supervisor Андреев Николай Владимирович, Университет ИТМО, факультет информационных технологий и программирования, заведующий лабораторией

Характеристика темы ВКР / Description of thesis subject (topic)

Название организации-партнера / Name of partner organization: Tiger Tech Limited
Тема в области фундаментальных исследований / Subject of fundamental research: нет / not
Тема в области прикладных исследований / Subject of applied research: да / yes

Основные вопросы, подлежащие разработке / Key issues to be analyzed
The main tasks to be solved are convenient workflow management, task setting and project execution control in a corporate environment. It helps
you easily assign tasks and track your progress. In the course of work on the WRC, it is necessary to solve the following tasks: 1.Description of the subject area consisting ,Problem analysis,Targeted Industry & Requirement Analysis, Functional and non functional Requirements and System Tech Stack 2.Software design consisting Designing the system architecture, data architecture and software architecture. 3. Implementation and testing including Software architecture, Development and Implementation of the user interface.

Форма представления материалов ВКР / Format(s) of thesis materials:
Operational processes in the form of a UML diagram, system architecture in the form of a diagram. Software architecture in the form of a diagram. Databases in the form
of a diagram. The program code. Screenshots of the implementation and testing.

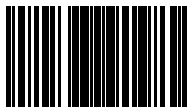
Дата выдачи задания / Assignment issued on: 25.01.2025

Срок представления готовой ВКР / Deadline for final edition of the thesis 15.05.2025

СОГЛАСОВАНО / AGREED:

Руководитель ВКР/ Thesis supervisor

Документ подписан
Андреев Николай Владимирович
16.05.2025

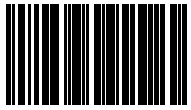


(эл. подпись)

Андреев Николай Владимирович

Задание принял к исполнению/ Objectives assumed BY

Документ подписан
Джахан Исрат
16.05.2025

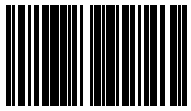


(эл. подпись)

Джахан Исрат

Руководитель ОП/ Head of educational program

Документ подписан
Маятин Александр Владимирович
20.05.2025



(эл. подпись)

Маятин Александр Владимирович

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

АННОТАЦИЯ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ SUMMARY OF A GRADUATION THESIS

Обучающийся / Student Джахан Исрат
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет информационных технологий и программирования
Группа/Group M34091
Направление подготовки/ Subject area 09.03.02 Информационные системы и технологии
Образовательная программа / Educational program Программирование и интернет-технологии 2021
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Тема ВКР/ Thesis topic Разработка модуля управления товарными запасами и продажами для ERP-системы / Development of Inventory and Sales Module for ERP System
Руководитель ВКР/ Thesis supervisor Андреев Николай Владимирович, Университет ИТМО, факультет информационных технологий и программирования, заведующий лабораторией

ХАРАКТЕРИСТИКА ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ DESCRIPTION OF THE GRADUATION THESIS

Цель исследования / Research goal
The Sales and Inventory Management System project's research objective is to assess the installation and effect of a complete digital system meant for effectively controlling sales and inventory processes in retail settings. The purpose of the study is to evaluate how well combining Django REST Framework, ReactJS, and PostgreSQL improves operational efficiency, user experience, and data correctness. Important goals are assessing performance measures, user interface usability, and inventory control and sales reporting automation. The findings will contribute to best practices in the deployment of sales and inventory management systems, offering insights into scalability, optimization, and the overall improvement of business operations in the retail sector.
Задачи, решаемые в ВКР / Research tasks
Analyze existing studies on sales and inventory systems and identify key trends, technologies, and methodologies. Define requirements and functionality through problem statements. Design and build the system using Django, ReactJS, and PostgreSQL. Create a database schema for efficient data. Testing the performance and functionality. Gather user feedback to assess usability. Analyze results for system effectiveness. Address areas for enhancement based on feedback and testing. Propose additional features and integrations for future development.
Краткая характеристика полученных результатов / Short summary of results/findings
Successful development and deployment of a Sales and Inventory Management System utilizing

Django REST Framework, ReactJS, and PostgreSQL. Validation of user interface effectiveness through Ant Design and enhancement of user experience. Automation of inventory and sales processes to improve operational efficiency. Implementation of a robust invoicing module for accurate financial management. Identification of scalability opportunities and areas for optimization based on user feedback.

Обучающийся/Student

Документ подписан
Джахан Исрат
20.05.2025

(эл. подпись/ signature)

Джахан Исрат
(Фамилия И.О./ name and surname)

Руководитель ВКР/
Thesis supervisor

Документ подписан
Андреев Николай Владимирович
20.05.2025

(эл. подпись/ signature)

Андреев Николай Владимирович
(Фамилия И.О./ name and surname)

Table of Contents

Table of Contents **1**
Definitions, generalizations and abbreviations **2**
Introduction **4**
1. Description of the subject area **5**
1.1. Problem analysis 5
1.2. Involved Processes 6
1.3. Functional requirements 9
1.3. Non-functional requirements 11
1.4. System Tech Stack 12
2. Software design **13**
2.1. System Architecture 13
2.2. Program architecture 14
2.2.1. Frontend Architecture 14
2.2.2. Backend Architecture 18
2.3. Data architecture 20
3. Implementation and testing **23**
3.1. Backend Use Case Implementation Example 23
3.2. Implementation of the user interface 25
Conclusion **34**
List of used literature **35**
Appendix A. Backend API routes **37**

Definitions, generalizations and abbreviations

ERP: ERP stands for Enterprise Resource Planning. It is a type of software system used by organizations to manage day-to-day business activities such as accounting, procurement, project management, risk management, compliance, and supply chain operations.

POS: The POS system is the place where sales are made. In a retail setting, it's the system used to complete customer purchases, process payments, and record transactions.

Inventory: Inventory refers to the goods and materials a business holds for the purpose of resale or production. Inventory management ensures the right products are in the right quantity at the right time.

API: API (Application Programming Interface) is a set of rules and protocols that allow different software applications to communicate with each other. APIs define how requests and responses should be structured so systems can exchange data in a standard way.

REST API: A REST API (Representational State Transfer) is a type of API that uses HTTP methods (GET, POST, PUT, DELETE) for interacting with resources. Django REST Framework (DRF) is a powerful and flexible toolkit built on Django to create RESTful APIs.

Django REST Framework: Django, a high-level Python web framework, is used to develop API for the Inventory and Sales Management. It provides essential tools, libraries, and pre-built structures to simplify the creation of dynamic, secure, and scalable web applications.

ORM: Django's built-in Object-Relational Mapping (ORM) is used to handle database interactions for the application. The ORM maps Python

objects to database tables, allowing developers to work with database records using Python code instead of raw SQL queries.

Introduction

The Sales and Inventory Management System is a comprehensive and user-friendly enterprise application developed as part of a thesis project, combining the power of Django REST Framework, ReactJS, and PostgreSQL to deliver a scalable and efficient solution for modern businesses. The frontend is designed with Ant Design, offering an intuitive and elegant user experience, while the backend is powered by robust RESTful APIs that ensure fast and secure data handling. This system is designed to simplify the end-to-end process of managing inventory and sales activities in small to medium-sized businesses. It enables users to manage product attributes, categorize products, and track their stock across various storage locations. Through a centralized dashboard, users can add new inventory items, update existing stock, and monitor inventory movements in real time. A key feature of the system is its invoicing module, which allows users to generate sales invoices dynamically by adding selected products to the cart, applying discounts, calculating taxes, and tracking payment statuses. The application ensures data consistency and accuracy across all modules using asynchronous communication between services and a modular microservice structure that supports easy maintenance and scalability. This project explores not only the technical implementation of a distributed ERP system but also the practical aspects of streamlining business processes like procurement, inventory control, and sales order management through a digital platform.

1. Description of the subject area

1.1. Problem analysis

In many small and medium-sized enterprises (SMEs), managing sales and inventory remains a significant challenge due to the lack of an integrated, automated system. Businesses often rely on manual record-keeping, spreadsheets, or disconnected software applications, which leads to issues such as stock mismanagement, data inconsistency, delayed decision-making, and human error. The absence of real-time visibility into stock levels and sales data often results in lost sales opportunities, overstocking, or understocking, all of which affect profitability and operational efficiency.

Furthermore, existing ERP solutions in the market are often complex, expensive, or overly generalized, making them unsuitable or inaccessible for smaller businesses with specific, localized needs. There is a clear gap in the availability of lightweight, customizable, and cost-effective ERP systems tailored for managing inventory and sales workflows in smaller business environments.

This thesis addresses the problem by designing and developing a web-based Sales and Inventory Management System built on Django REST Framework and ReactJS. The goal is to provide a user-friendly, modular, and scalable solution that streamlines core business processes such as product tracking, inventory control, invoicing, and reporting.

This inventory and sales management system is mainly targeted for lifestyle-type businesses, for example, the clothing industry, fashion industry, supermarkets, food chains, ornaments, and tech shops.

Industries are growing day by day, so they want their products to be in a managed system where they can store their information. Also, the customers want to get their item as soon as possible. They want a quick sellout service. So automation is necessary. That's where this POS system comes in handy.

1.2. Involved Processes

Figure 1 illustrates the user authentication process. This diagram visually represents the sequence of actions and decision points that occur when a user attempts to log in to the system.

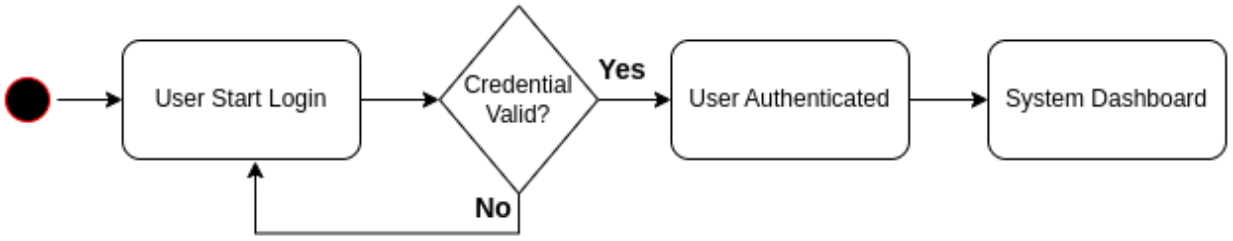


Figure 1. User authentication action diagram

The process begins when a user initiates the login procedure. Once the login process is started, the system prompts the user to enter their credentials (typically a username and password). The process then moves to a decision point labeled "Credential Valid?" where the system checks the validity of the provided credentials. If credentials are valid, the user gets access; if not, it will give the user to correct the credentials. If still not valid, it won't allow the access.

The inventory module diagram (Figure 2) demonstrates the workflow for managing inventory within the Sales and Inventory Management System.

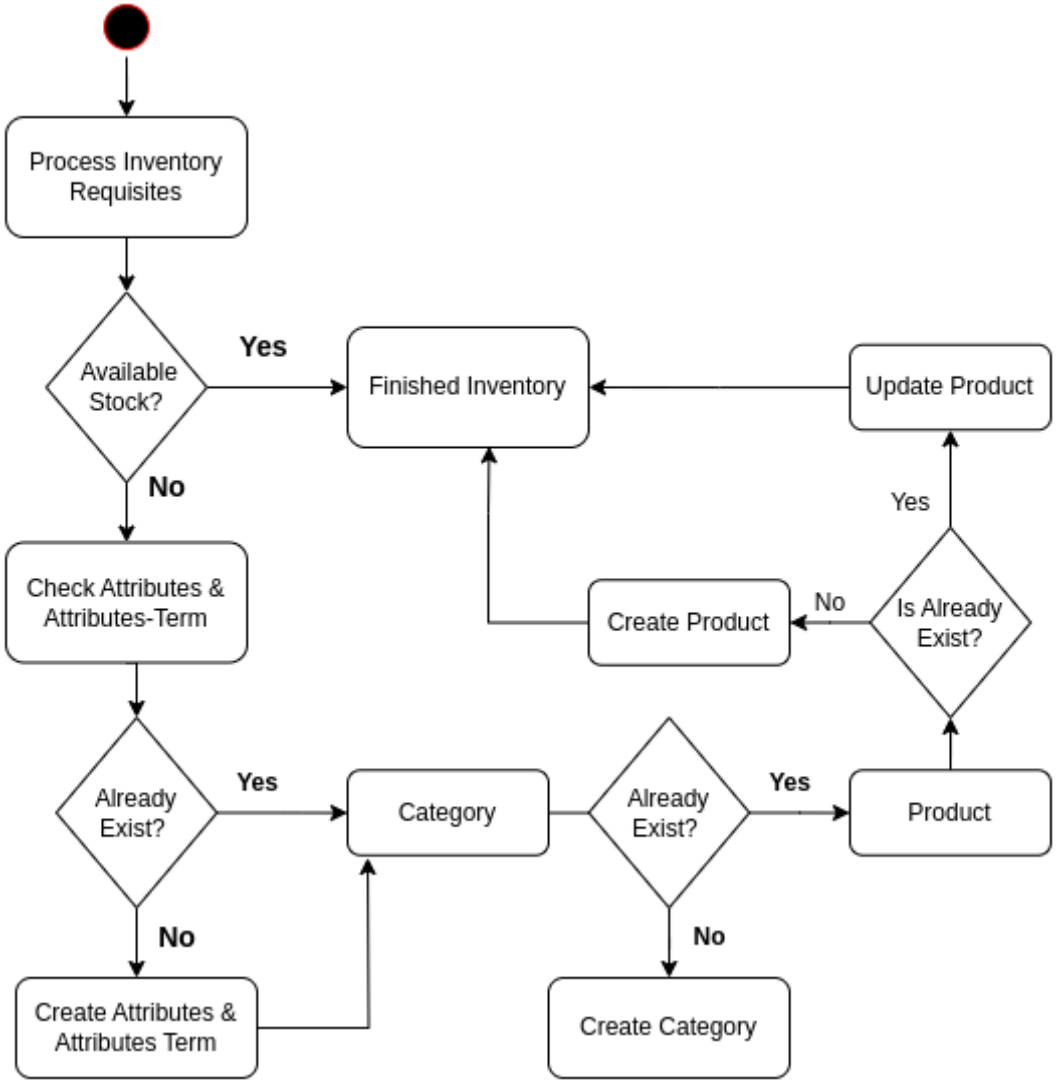


Figure 2. Inventory module diagram

The process begins with handling inventory requisites and checking for available stock. If stock is available, the inventory is marked as finished; otherwise, the system checks product attributes and terms. If the required attributes do not exist, they are created before proceeding. The workflow also includes verifying and creating categories and products as needed, ensuring that duplicate entries are avoided. When a product already exists, it is updated; otherwise, a new product is created and associated with the appropriate category. This structured approach ensures efficient inventory tracking, prevents redundancy, and maintains data integrity throughout the inventory management process.

The sales module action diagram (Figure 3) depicts the step-by-step workflow for processing a sales transaction in the system.

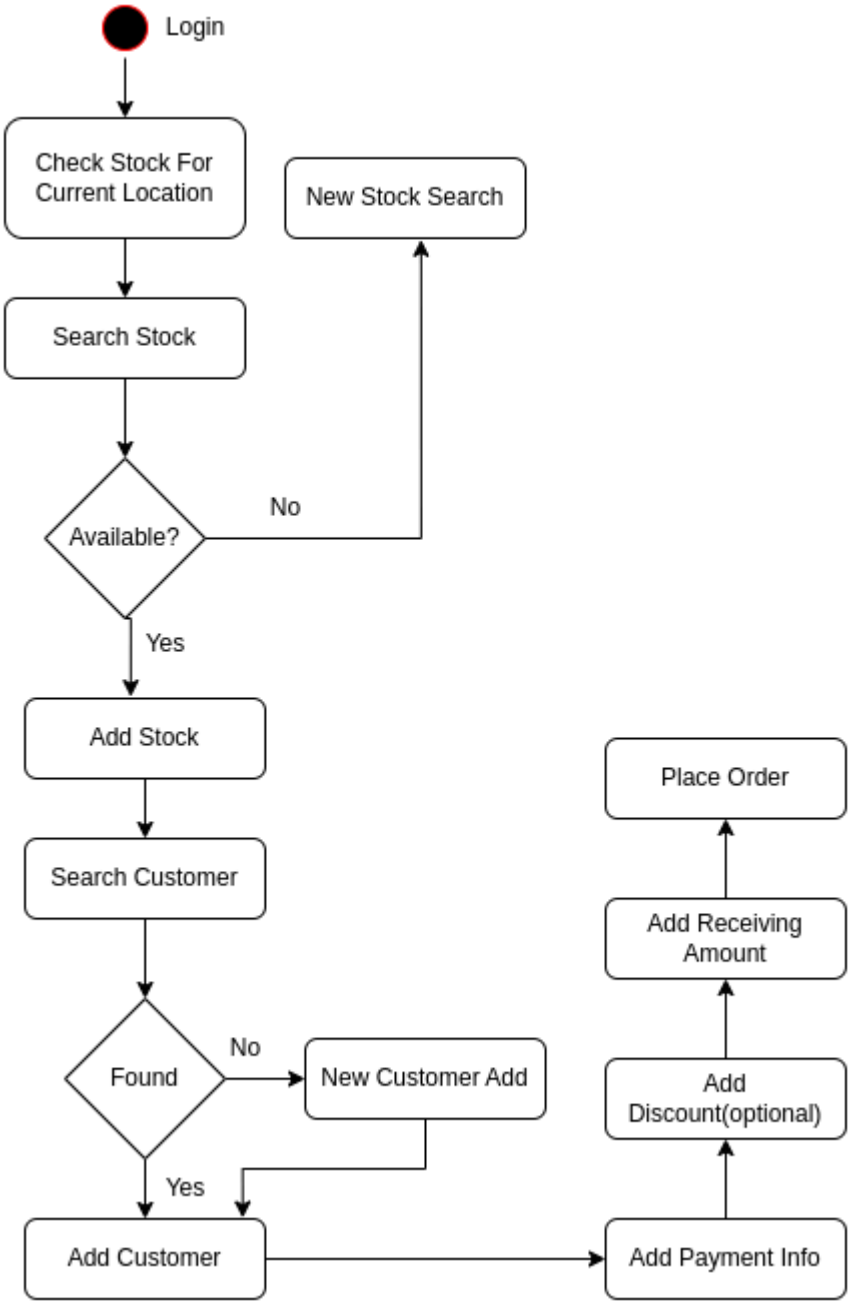


Figure 3. User authentication action diagram

The process begins with user login, followed by checking stock availability at the current location. If the desired stock is available, it is added to the transaction; if not, a new stock search is initiated. Next, the system searches for the customer; if the customer is not found, a new customer record is created. Once the

customer is added, and payment information is entered, with options to include discounts. The transaction concludes with the addition of the received amount and the final placement of the order. This structured flow ensures that each sales transaction is accurately processed, linking inventory, customer data, and payment details for efficient and reliable sales management.

1.3. Functional requirements

In this Inventory and Sales Management System, the main actors are the admin and the salesperson. They are the ones who manage the system, put up stocks, and sell them to customers offline. Here, customers may be a part of actors if the sales occur online, but as the system is managed for offline businesses, the customers are out of the actor area.

From the processes above and from further discussions with customers following functional requirements were identified:

- User/Employee Creation: In the software, users can be created by adding Department, Designation and then assigning those departments and designation to the newly created Employee. The user will have specific privileges to the assigned system.
- Attributes for Inventory: The inventory may consist of attributes/variations that should be added before. For our system, color, size, fabrics, materials, etc. can be added to the system for lifestyle businesses. The attribute terms consist of the specific attribute.
- Category Of Inventory: The category makes the inventory more appropriate. The product or item will be under a category. Multiple categories can be created with a parent-child system to showcase more subcategories.
- Product: This is the main part of an Inventory. When creating a new product, some items must be input: Category of the product, Purchase price, Selling

Price, Location of entry, supplier etc. If the product has variation like color/size, the quantity is added for that variation. After adding a product, the barcode is auto-created and can be printed as well.

- Sales Panel/POS: The sales person has access to this panel where the product is sold to customers. Here in POS, some input is a must, like the location of sales panel, Customer information, product variation available at that location, discount(if givable), and payment info. After that, the salesperson can work on an Exchange product and update any order that is placed.

- Customer Record: Customer info is added when a product is sold. For any customer related issue or checking all purchases of a customer, customer panel can be used to check them out and download excel details of them.

- Reporting Panel: In the Reporting Module, all the reports of the system can be viewed. Here we have an *All Product Entry Report* where all product entries with their info are added with date time. There is also a *Stock Report* where all available stocks can be viewed with category and location. There is also a *Sales Report* where all sales records, invoices and outlet/location wise records are stored. Then finally *Sold Product Report* where the sold record is shown with customer info, product variation sold, sold price etc.

Main functional requirements are presented on Figure 4.

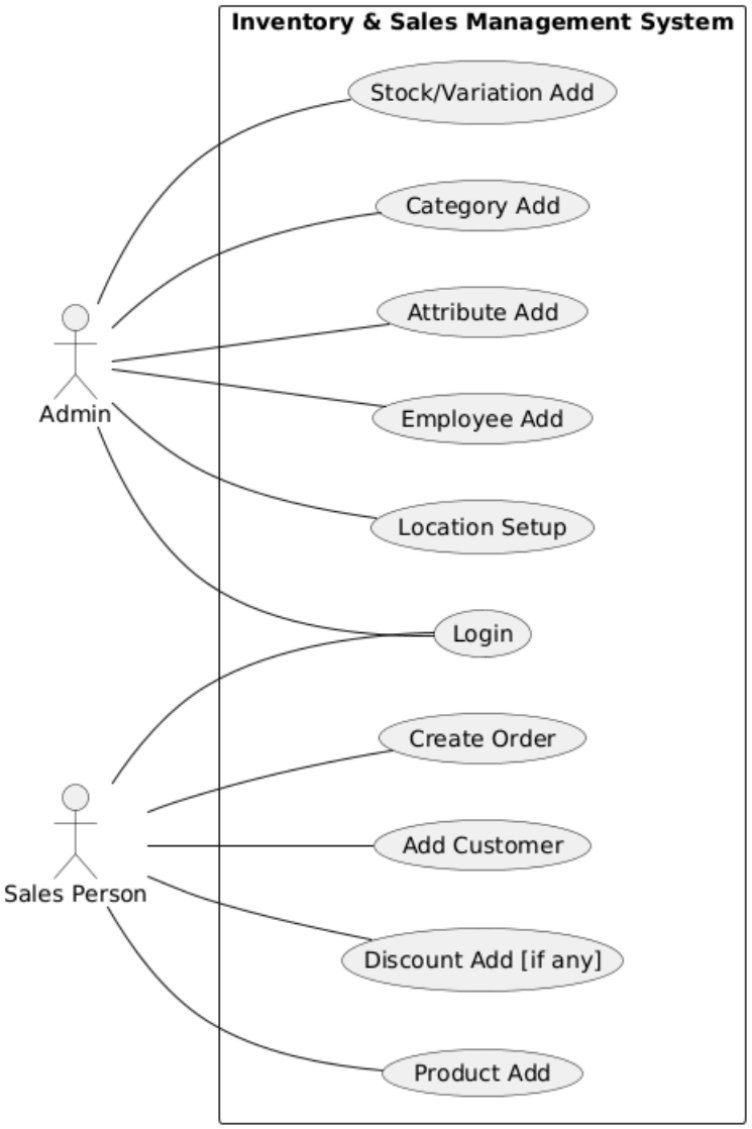


Figure 4: Use Case Diagram for Inventory and Sales Management System

1.3. Non-functional requirements

Alongside functional requirements, non-functional ones were set:

- Performance:
  - The system should handle up to 10,000 inventory records without significant performance degradation.
  - Load reports within 5-10 seconds for up to 1,000 transactions.
- Scalability:
  - The system should support the addition of new locations, products,

categories, and modules without major architectural changes.

- Security:
    - Role-based access control to restrict access to specific modules and data.
    - Secure sensitive data (e.g., customer details, supplier information) using encryption.
- Maintainability:
    - Modular code structure to allow independent updates to specific features (e.g., inventory, sales).
    - Documentation for APIs, database schemas, and user manuals.
- Interoperability :
    - Export data in standard formats (e.g., Excel, PDF) for external reporting tools.
- Data Integrity:
    - Prevent duplication or loss of data during stock transfers, payments, and updates.

1.4. System Tech Stack

This web application system is designed to work from any browser. It is an online system, so it is easily accessed and updated.

- Frontend Development: ReactJS and Ant Design;
- Backend API Development: Python and Django REST Framework;
- Database management system: PostgreSQL;
- Deployment: Vercel for frontend and Railway for backend;
- Version control system: Git and GitHub.

2. Software design

2.1. System Architecture

The deployment diagram shown in Figure 5 shows a detailed three-tier setup used for this project.

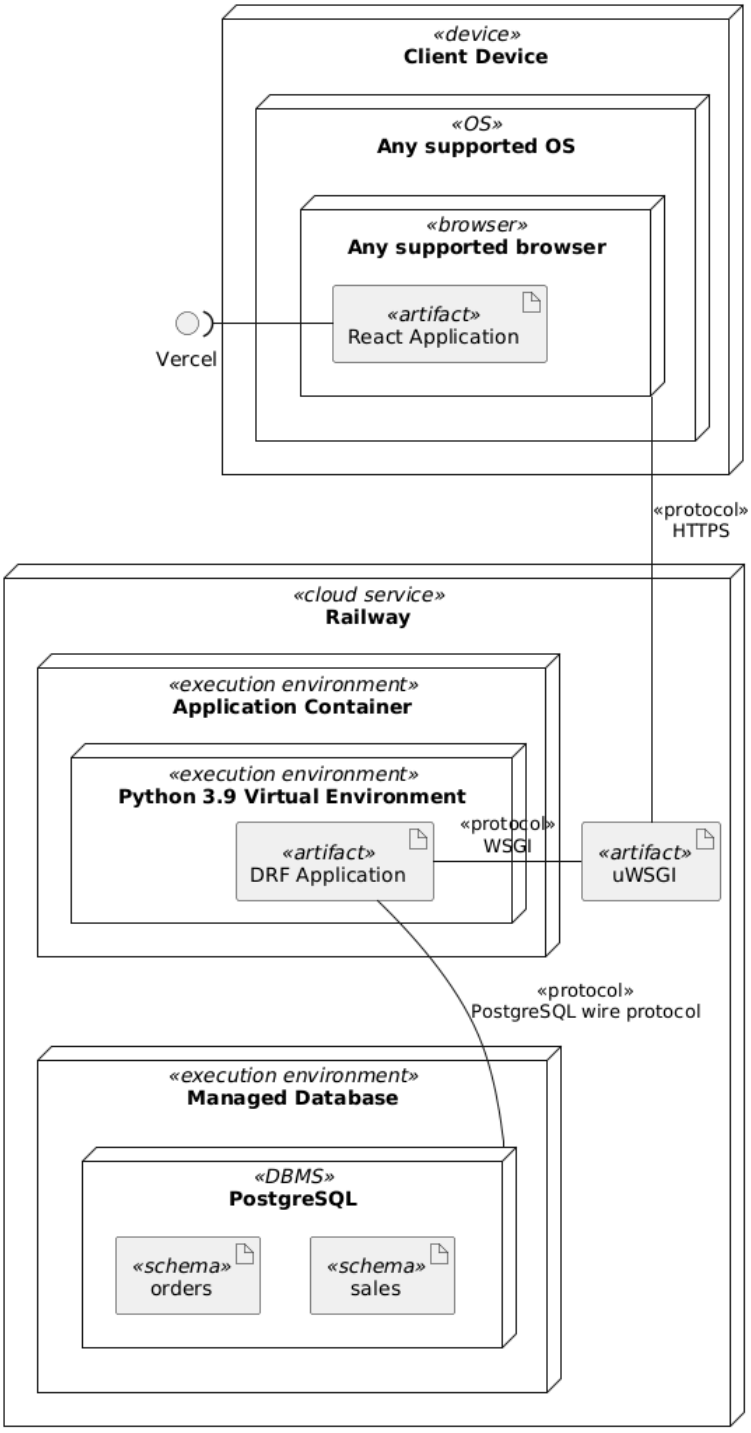


Figure 5. Deployment diagram

The frontend layer is hosted on Vercel, a cloud service designed for deploying static sites and serverless functions. Vercel is primarily used for frontend building and hosting; thus, its internal architecture is not described on this diagram.

The backend layer is hosted on Railway. It includes the Django REST Framework API that acts as the core logic. It consists of different modules like Inventory Management for tracking products and stock, Sales and POS for handling transactions, Human Resource Management for managing employee info, Customer Relationship Management for keeping customer data, and a Reporting Module for generating analytics.

The database layer utilizes PostgreSQL hosted on Railway Cloud infrastructure. It stores all module data, maintaining persistent information for all microservices.

2.2. Program architecture

Because both the backend and frontend were developed in this work, they will be discussed separately.

2.2.1. Frontend Architecture

Top-level program architecture for the frontend is shown in Figure 6. We can see that the primary package with all related source code is the src package, which contains various subpackages. Briefly describe these subpackages:

components: This is where all the building blocks of the user interface live. Each folder inside components — such as Contact, Product, Order, HRM, Dashboard, and Purchase — represents a distinct feature or section of the application. By grouping related React components together, development becomes more organized, and it's easier to maintain or expand individual features without affecting others.

actions: The actions folder is home to the logic that communicates with the

backend. Whenever the frontend needs to fetch, update, or delete data, it does so through these actions. This separation ensures that the user interface remains clean and focused on presentation, while all the data-handling logic is centralized here.

api: All the details about how the frontend talks to external services are tucked away in the api folder. This includes setting up HTTP requests, managing endpoints, and handling things like authentication. Keeping this logic in one place makes it simple to update or debug how the app connects to the backend.

redux: The redux folder manages the application's state. Here you'll find reducers, actions, and other files that keep track of data shared across different parts of the app. By isolating state management, the codebase stays organized, and it's easier to understand how data flows throughout the application.

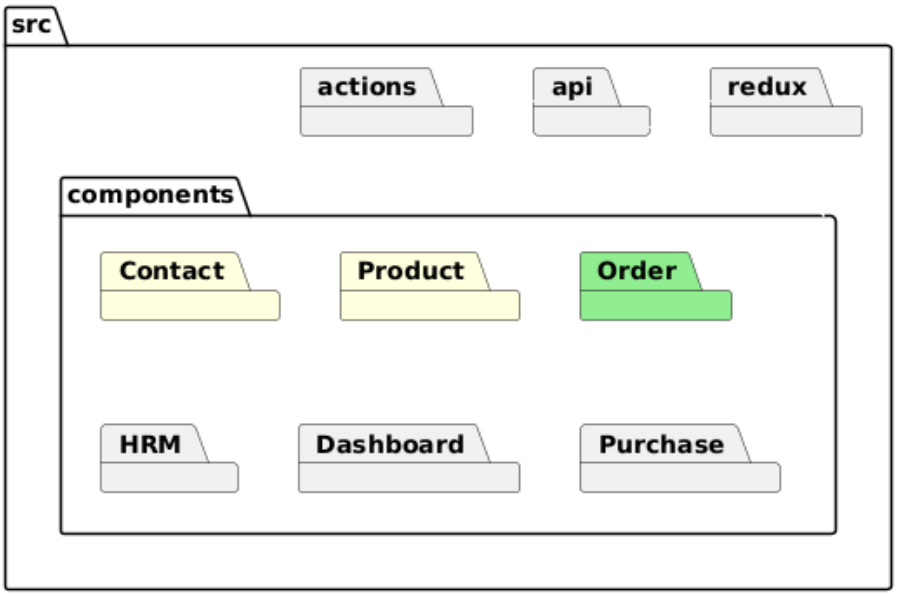


Figure 6. Top-level frontend architecture

Main packages that have to be developed are components/Contact/Customer, components/Product/ProductDetails, and components/Order. Describe these packages in depth.

Package components/Contact/Customer contains customer management-related components, such as viewing and editing customer contact

details (ContactDetails, EditContact), creating new contacts (CreateNewContact), downloading PDFs (PdfDownload), and rendering tables of customer data (RenderTable). They are exposed to the main application using a centralized index file. Component diagram for this package is shown in Figure 7.

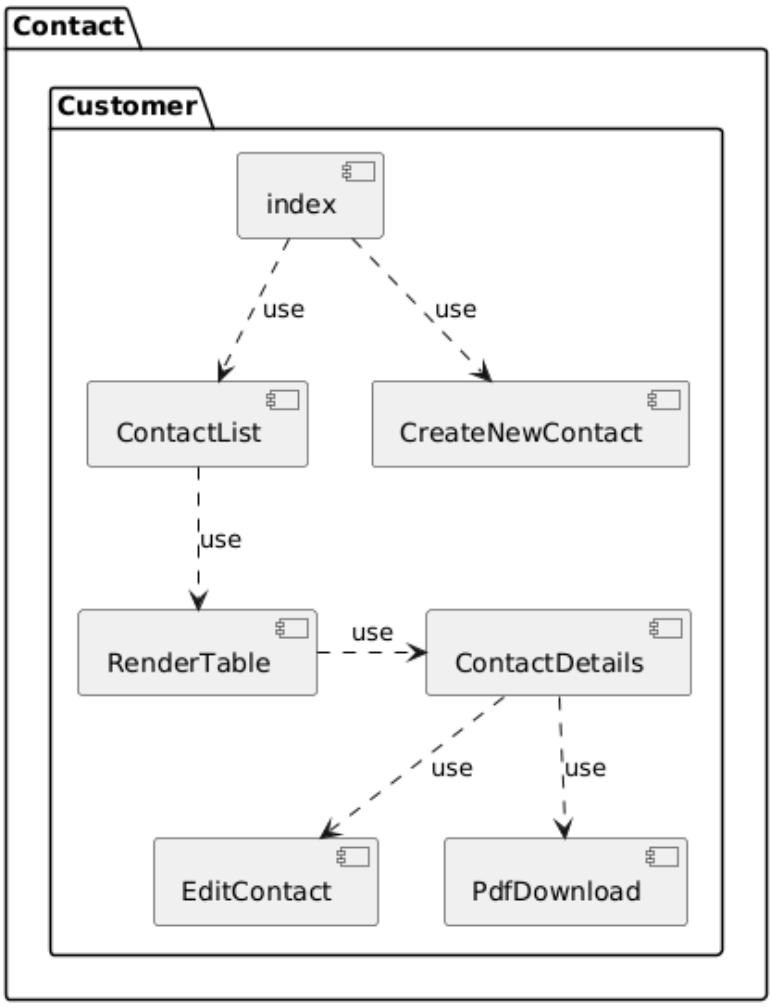


Figure 7. Program architecture for components/Contact/Customer package on frontend

Package components/Product/ProductDetails contains detailed product management components, such as listing all products (AllProductList), editing product details (EditProduct, EditVariableProduct), uploading images (ImageUpload), managing product pricing and variations (CountPrice, CountVariation), rendering product data (RenderTable) and additional utilities like product logs, quick views, and demo files. Component diagram for this package

with some components is described in Figure 8.

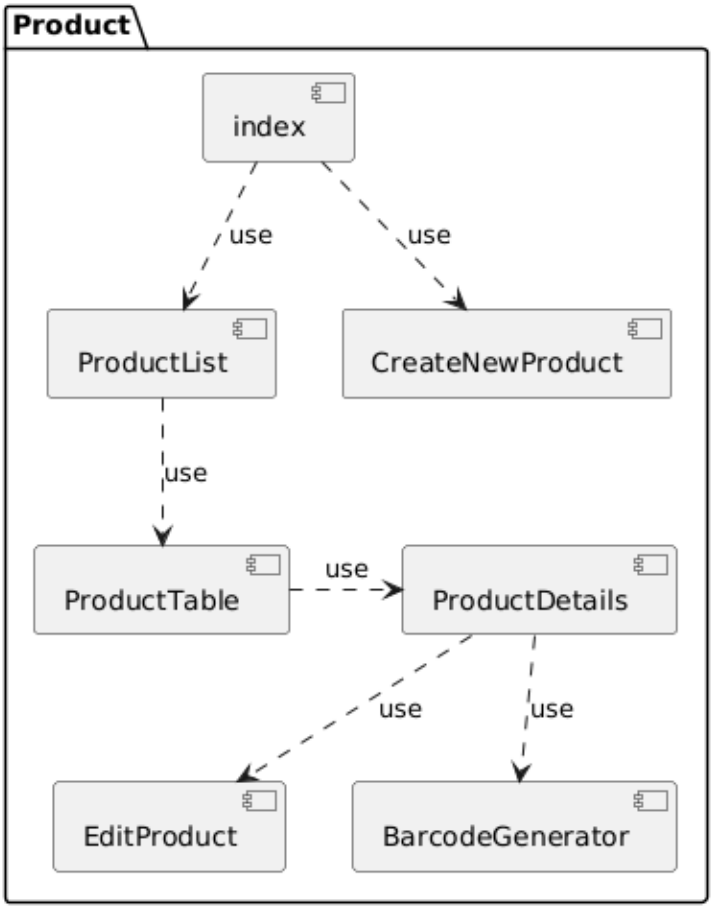


Figure 8. Program architecture for components/Product/ProductDetails package on frontend

Package components/Order manages order related activity: components for handling invoices, cost sheets, order exchanges, and both new and old orders, such as: adding and updating packaging, services, shipping, and measurements, cancelling orders (CancelOrder), creating and editing contacts and measurements, generating invoices (GenerateInvoice), viewing payment history and memory, quick view functionality for rapid access to order data. Component diagram for this package with some components is described in Figure 9.

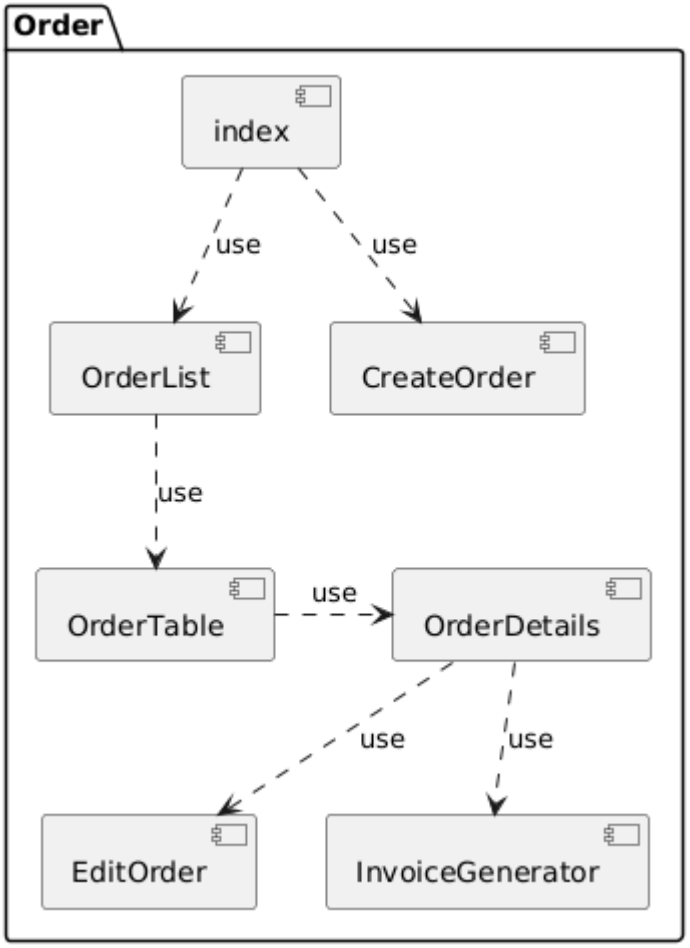


Figure 9. Program architecture for components/Order package on frontend

2.2.2. Backend Architecture

The backend source code for the Sales and Inventory Management System website was implemented using the Django web framework. This structure demonstrates a modular approach, where each core business domain is encapsulated within its own directory under the main project folder.

Main project folder contains the following components:

- settings.py: main library for project configuration; all middleware, environment setup, and installed apps are configured here.
- .env: environment file for database credentials and secret keys.
- urls.py: main routing file where admin routes and app routes are declared.

- asgi.py and wsgi.py: These are entry-point files used to serve your application. They are interfaces between Django code and the web server.

Each domain-specific package in this project contains the following key files and subpackages:

- __pycache__/: It is generated by Python automatically to store compiled bytecode to improve performance.
- migrations/: Migration includes scripts that track and apply changes to the database schema, protecting data consistency as the system evolves.
- __init__.py: Label the directory as a Python package, allowing for modular imports and code organization.
- admin.py: Facilitates the registration of models within Django's integrated administrative interface, allowing website administrators to oversee data via a visual dashboard.
- apps.py: Contains configuration for each functional area to help Django recognize and integrate into the overall project.
- models.py: Establishes data structures for each area, such as products and orders, that are necessary for inventory and sales management.
- permissions.py: To ensure that only authorized users can perform certain actions, such as editing products, user-defined access control logic is implemented.
- serializers.py: It facilitates the conversion of complex data types (for example, model instances) to JSON format and back, ensuring uninterrupted communication between the server side and the website interface using the RESTful API.
- signals.py: Contains event-driven logic that responds to changes in the database, for example, updates inventory data when a new order is placed.
- tests.py: To verify the correctness and reliability of the internal logic, automated tests are provided, increasing the reliability of the website.

- urls.py: Maps URL templates to view specific functions or classes, defining API endpoints through which the interface interacts with the backend.
- views.py: This component performs the fundamental business logic needed to process HTTP requests and formulate responses, including creating new orders and listing available products.

Program architecture for one of the domain-specific packages is presented on Figure 10.

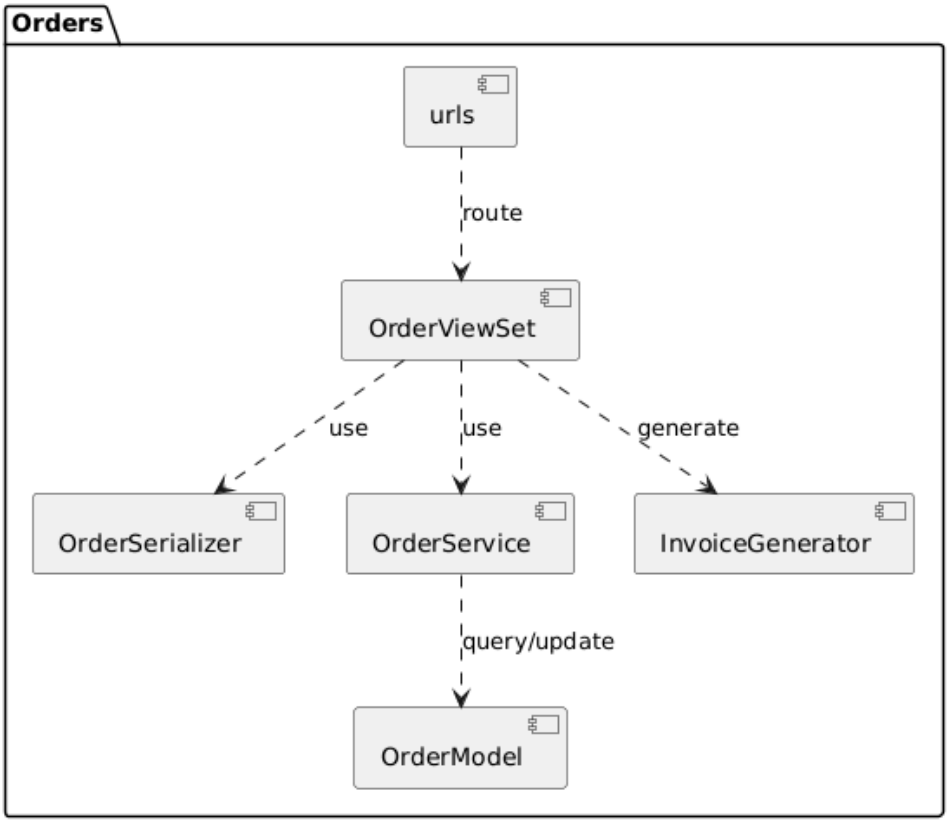


Figure 10. Program architecture for orders package on backend

2.3. Data architecture

The schema diagram in Figure 11 presents the relational structure of the database. It illustrates how key entities — including Customer, Category, Invoice, Attribute, and others — are organized and interconnected. Each table contains fields relevant to its domain (such as product pricing, customer contact information, invoice details, and so on) and uses foreign key relationships to maintain data integrity and ensure efficient querying.

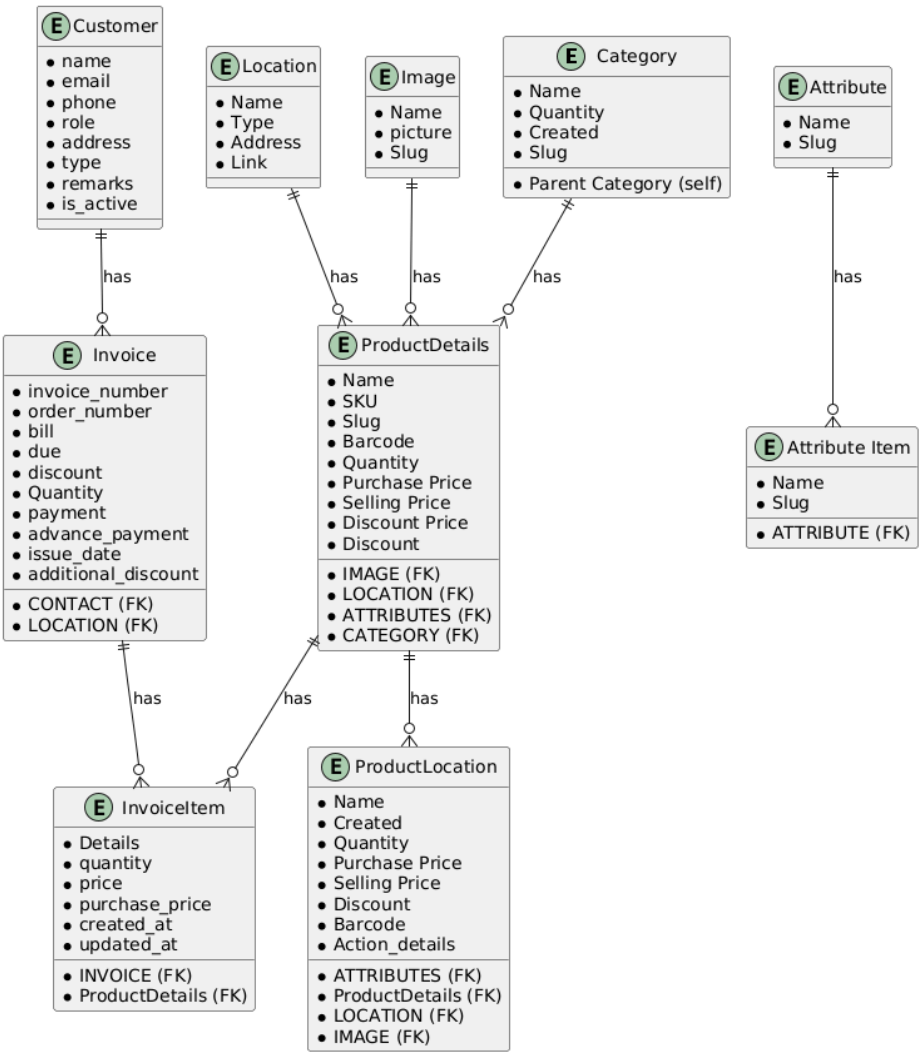


Figure 11. Schema of database tables

Customer model represents individuals or organizations that make purchases or orders. Stores contact details, address, role, and status information. Location model defines physical or virtual places associated with products or transactions. Includes name, type, address, and a link for reference.

Image model stores information about product images. Each record contains the image name, file reference, and a slug for identification.

Category model organizes products into hierarchical groups for easier management and browsing. Contains category details and quantity and supports parent-child relationships.

Attribute model defines characteristics or properties that can be assigned to

products, such as color or size. Each attribute has a name and a unique slug. AttributeItem model represents specific values for a given attribute (e.g., "Red" for the "Color" attribute). Links each value to its parent attribute.

Invoice model documents sales transactions made by customers. Records billing details, payment status, discounts, and links to customers and locations. InvoiceItem model lists individual products or services included in an invoice. Captures quantity and pricing and references the related product and invoice.

ProductDetails model stores core information about each product, including SKU, pricing, quantity, and associated images, categories, and attributes. Serves as the main product catalog entry. ProductLocation model tracks the presence and status of products at specific locations. Includes inventory details, pricing, and links to product attributes, images, and locations.

3. Implementation and testing

3.1. Backend Use Case Implementation Example

The UML sequence diagram on Figure 12 illustrates the process of creating a new product within a product management system from the perspective of an administrative user (Admin). The diagram captures the flow of control and data between key system components as a product is added to the system.

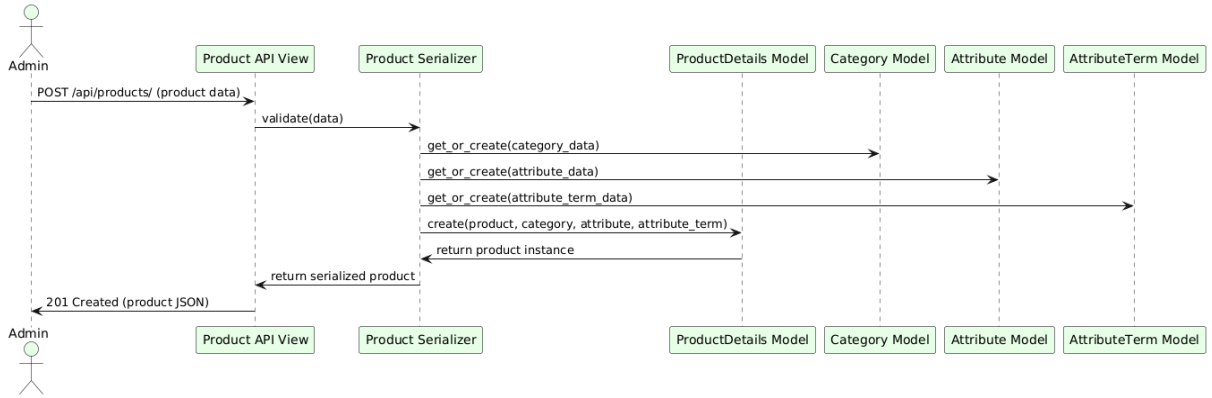


Figure 12. Sequence diagram Product module

The process begins when the Admin submits product data through the Product API View, which acts as the entry point for product creation requests. The Product API View forwards this data to the Product Serializer, responsible for validating and preparing the data for further processing. The Product Serializer then coordinates the retrieval or creation of related entities: it interacts with the Category Model to ensure the product is assigned to the correct category, with the Attribute Model to attach relevant product attributes, and with the AttributeTerm Model to specify attribute values.

Once all necessary associations are established, the Product Serializer instructs the ProductDetails Model to create the new product, incorporating the provided data and relationships. The ProductDetails Model returns the newly created product instance to the Product Serializer, which serializes the result and sends it back to the Product API View. Finally, the Product API View returns a

confirmation response to the Admin, signaling the successful creation of the product.

This sequence diagram demonstrates how the system ensures data integrity and consistency by coordinating the creation and association of product, category, attribute, and attribute term entities in a clear, traceable sequence. It serves as a valuable tool for understanding and documenting the dynamic behavior of the product management process in a software system.

Next UML sequence diagram on Figure 13 models the workflow of an order processing system within an inventory and sales management application. The diagram visually sequences the key interactions among system components that occur when a salesperson initiates an order. The participants, listed from left to right as ProductDetails Model, Invoice Model, Customer Model, Order API View, Order Serializer, and InvoiceItem Model, represent the main functional units involved in processing a customer order.

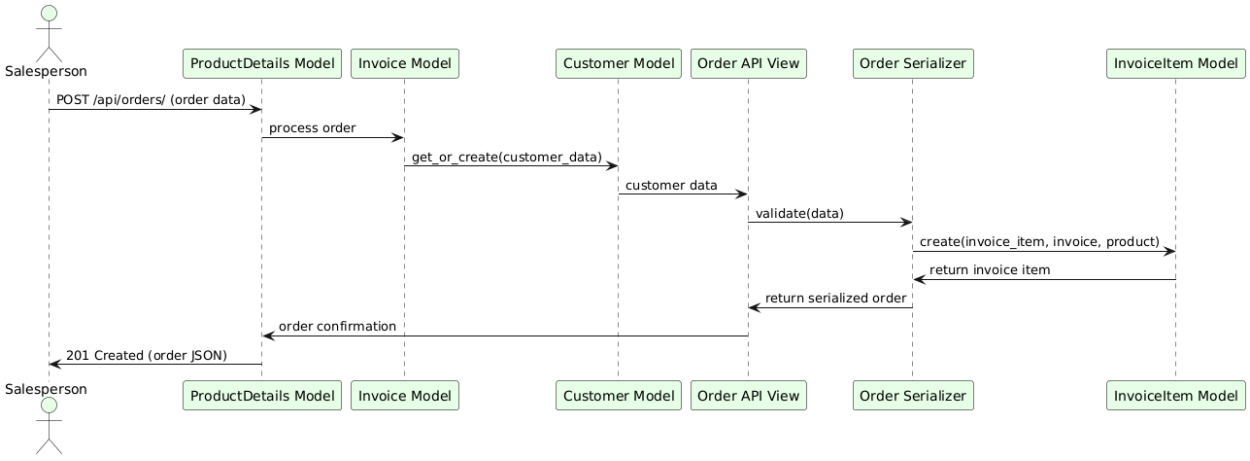


Figure 13. Sequence diagram Order module

The sequence begins with the salesperson submitting order data. The ProductDetails Model receives the order and initiates the process, followed by the Invoice Model, which manages the creation and tracking of invoices. The Customer Model is responsible for retrieving or creating customer records as

needed. The Order API View and Order Serializer handle request validation, data serialization, and orchestration between components. Finally, the InvoiceItem Model is responsible for associating products with invoices, ensuring each item in the order is properly recorded.

This structured approach ensures that all necessary business logic — such as customer validation, invoice creation, and item association — is executed in a clear, traceable sequence. The diagram highlights the importance of each component in maintaining data integrity, supporting real-time inventory updates, and facilitating smooth communication between the user interface and backend systems.

Implemented API routes are described in Appendix A for reference.

3.2. Implementation of the user interface

This is the dashboard after login. The dashboard provides a comprehensive overview of inventory levels, sales statistics, and recent activities. It serves as the central hub for users to quickly access essential modules and monitor business performance in real time.

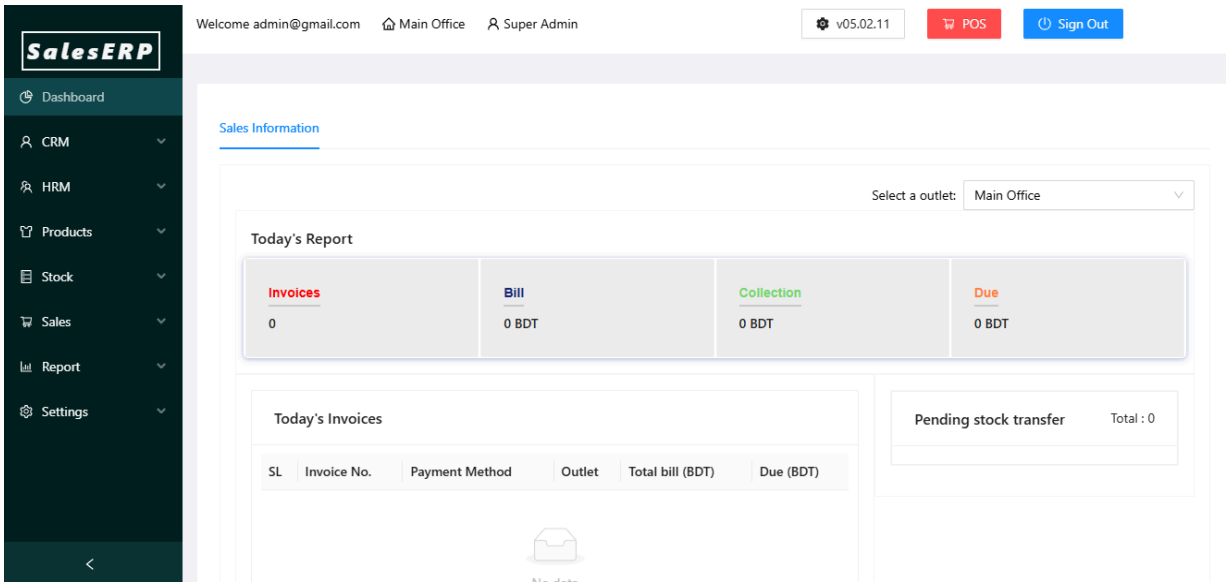


Figure 14. Dashboard of interface

It is under the Product Module. Here attributes can be added, and after creating attributes, their attribute terms can be added as well.

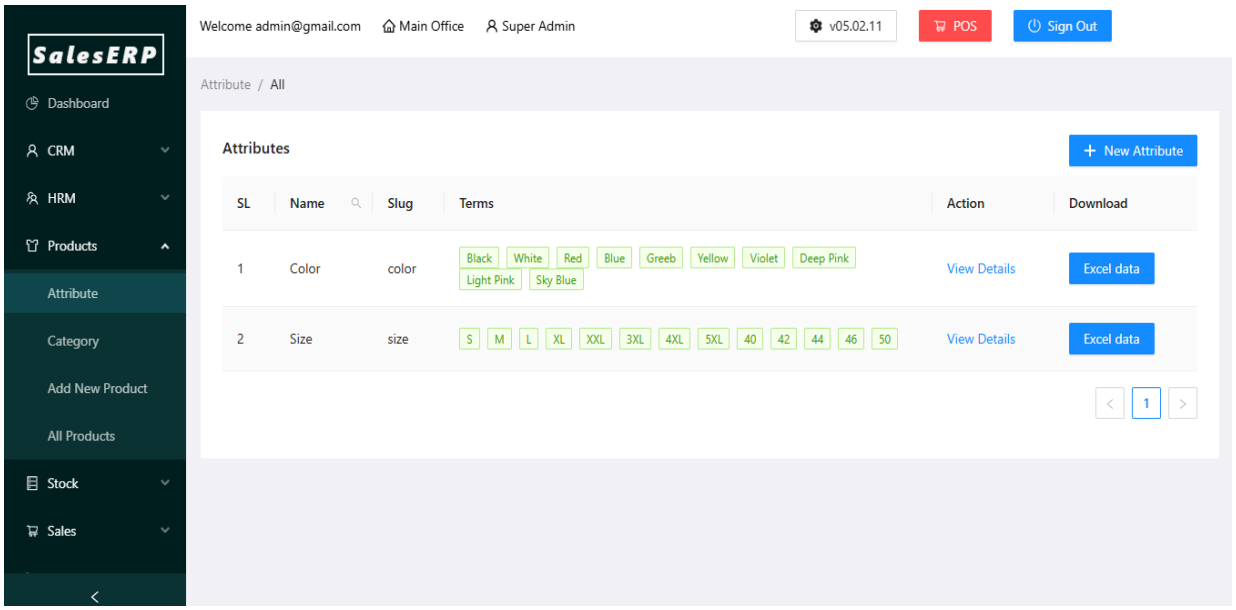


Figure 15. Attribute module

In this submodule of Product, Category can be added. Also, if the category has a parent attribute, that can be added as well. that way the child category will be under the parent category attribute.

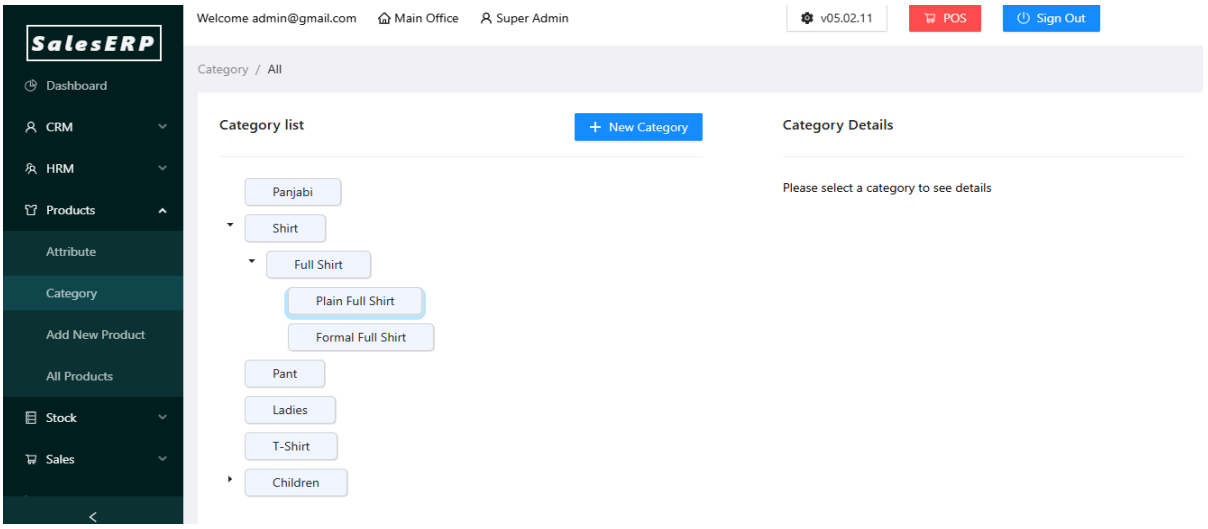


Figure 16. Category module

This is the main submodule for adding stock. Here, a product is added with their attribute-wise variation, like color, size, or quantity. Also, the product has its

category, location for the stock, cost, and unique code.

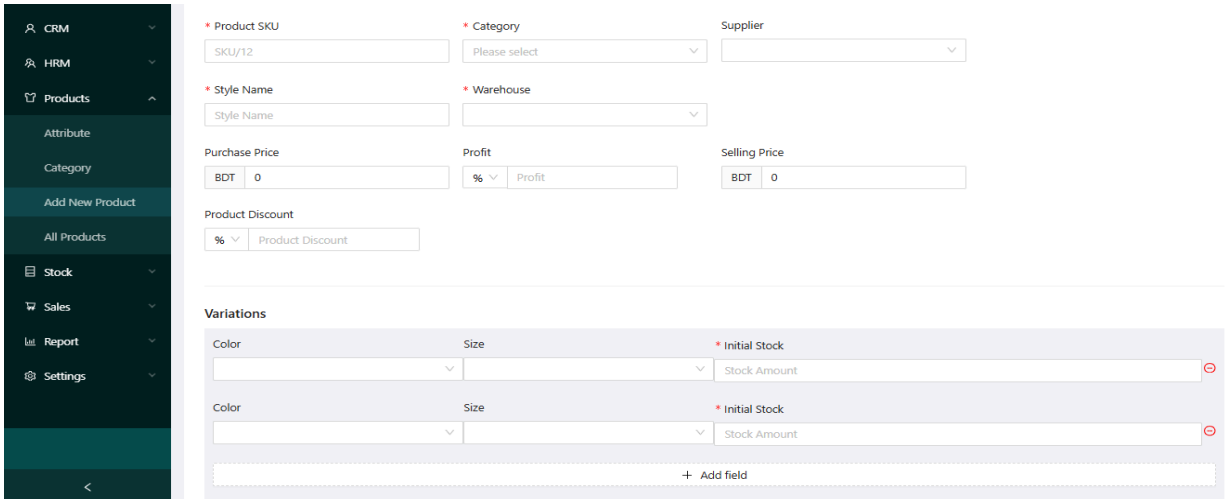


Figure 17. Product module

After a product is created, it will be shown in the all product section. In this section, the product will have a barcode that is created combined with variation and product ID. Also, the location product is located here.

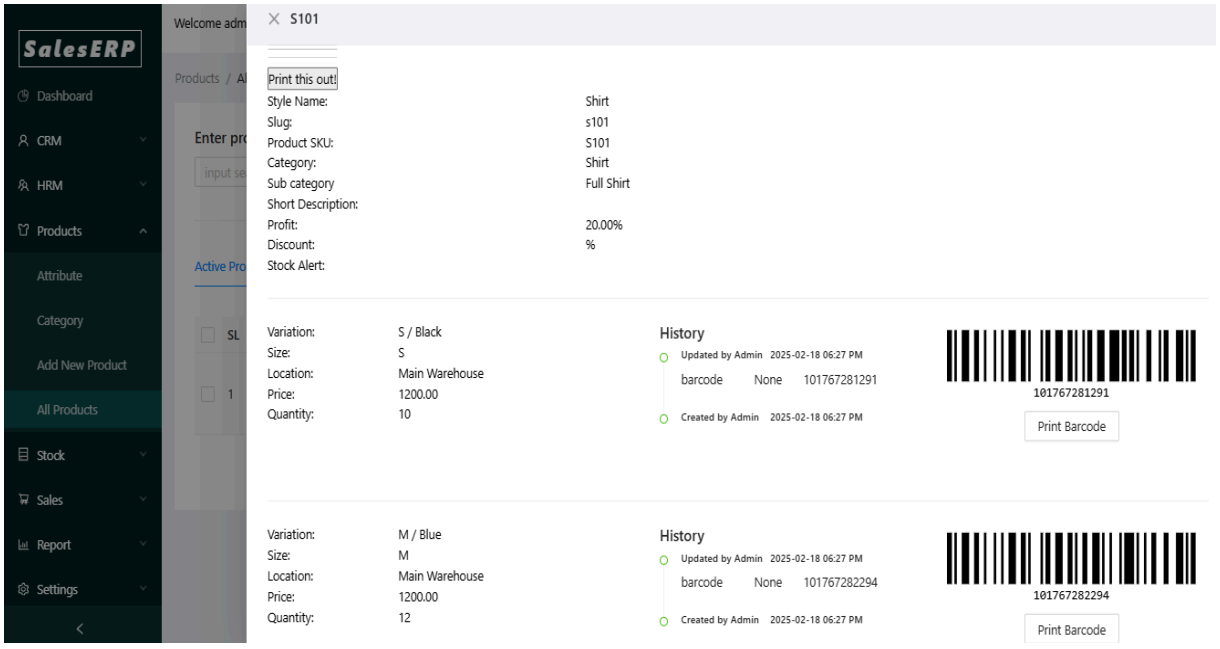


Figure 18. All Product module

This page displays the current stock levels of all products that have been added to the inventory. Users can view available quantities, product details and

quickly assess which items need restocking or attention.

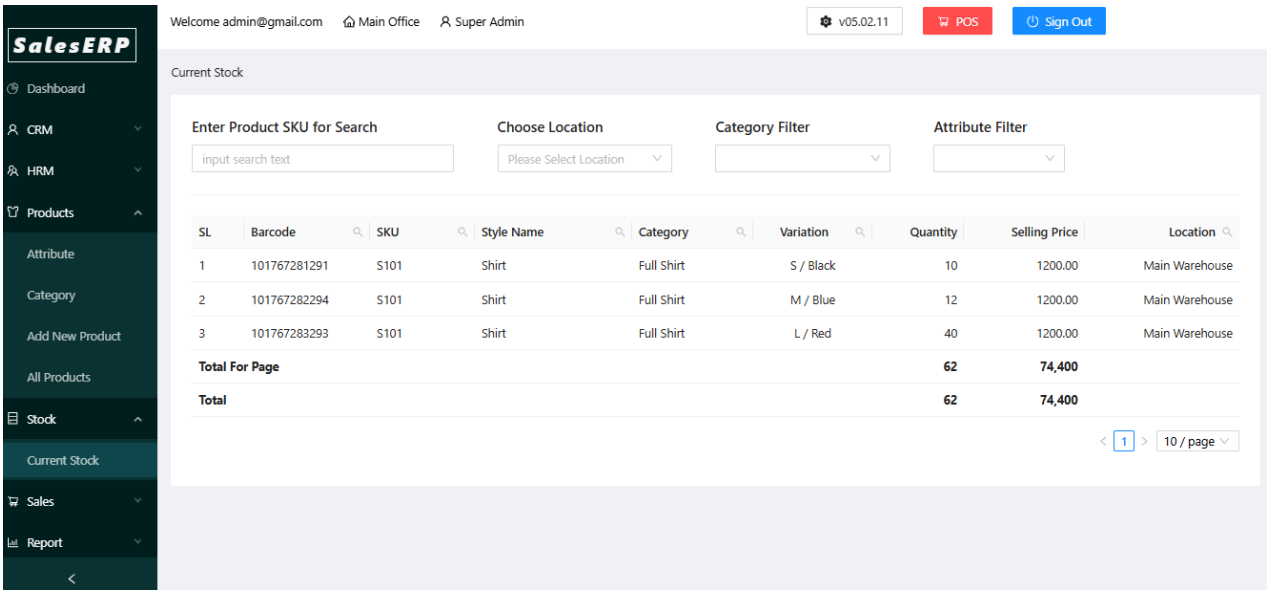


Figure 19. Current stock module

This is the main sales portal. Here, stock can be added via the location it is situated. Then a customer is added where the phone number is primary. Sales person can also add a discount if it applies. Clicking Confirm Order will create an order and print an invoice if it is connected with a POS machine.

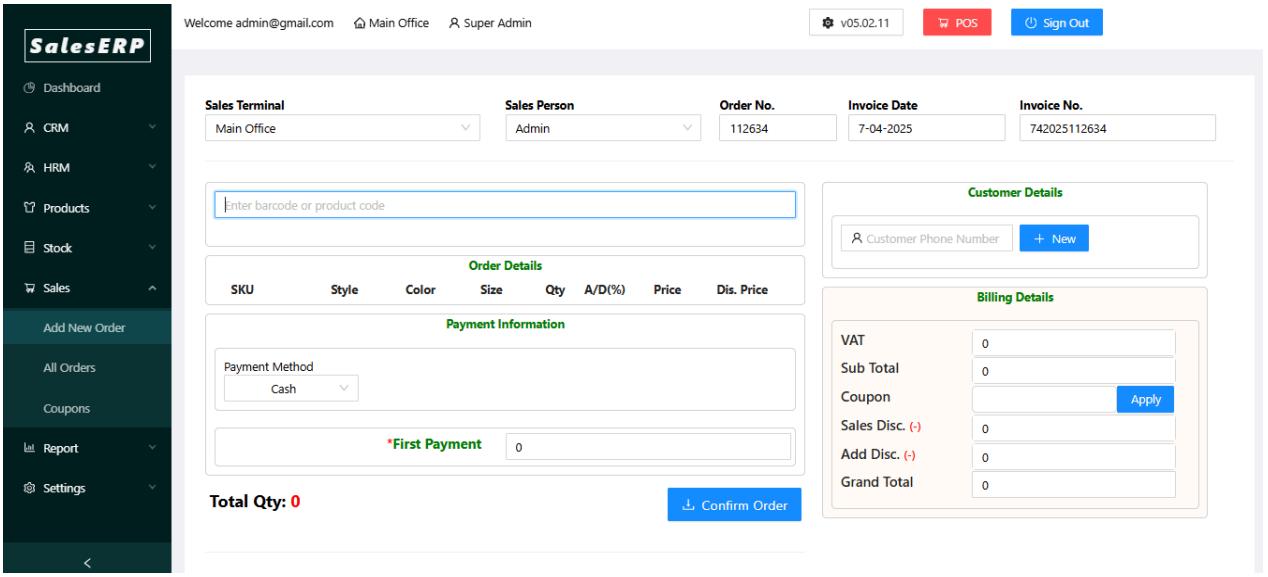


Figure 20. POS/Add New Order

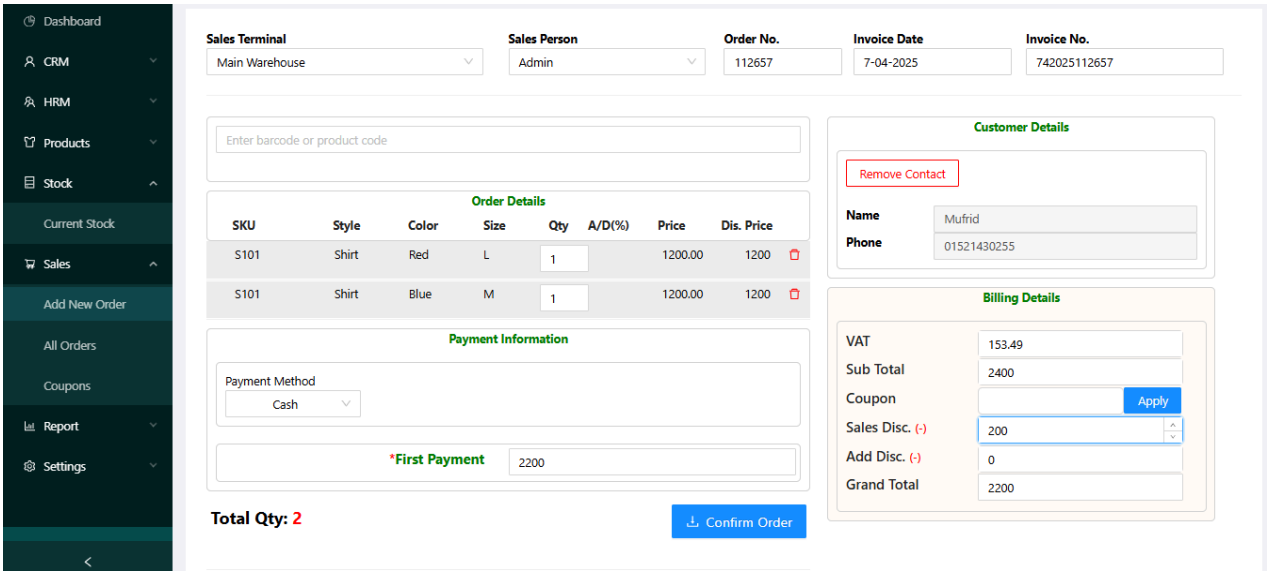


Figure 21. POS/Add New Order

User can check sales reports based on date and outlet location filter. It helps management track sales performance across different periods and store branches for informed decision-making.

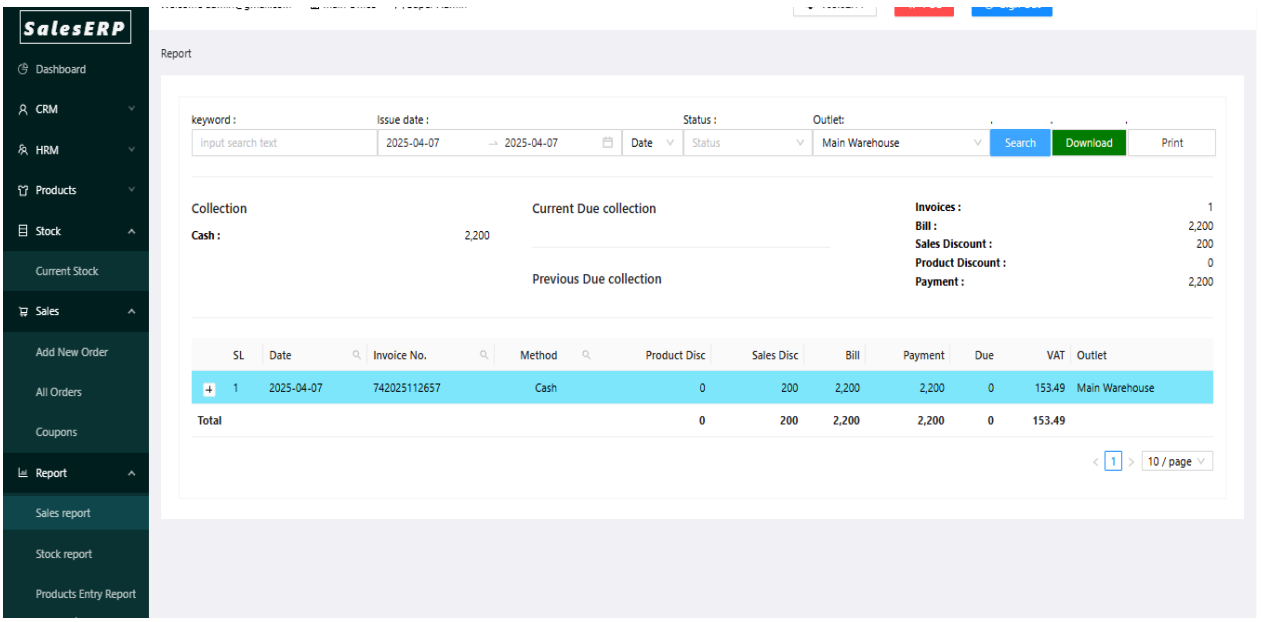


Figure 22. Sales Report module

Current stock report of an All/single location. Excel reports can be downloaded here. Users can view real-time inventory data and download detailed Excel reports for further analysis or record-keeping.

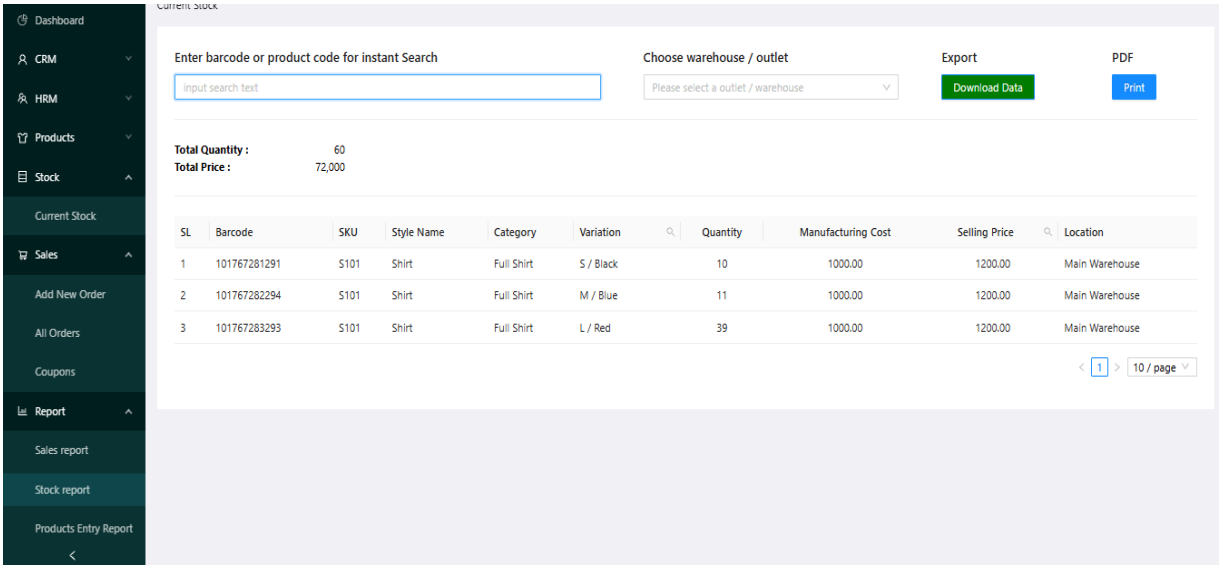


Figure 23. Stock report module

This is the report of sold items that customers bought. It can be filtered date-wise including details such as product, quantity, and buyer information. Users can filter the report by date to analyze sales trends and customer purchasing behavior over specific periods.

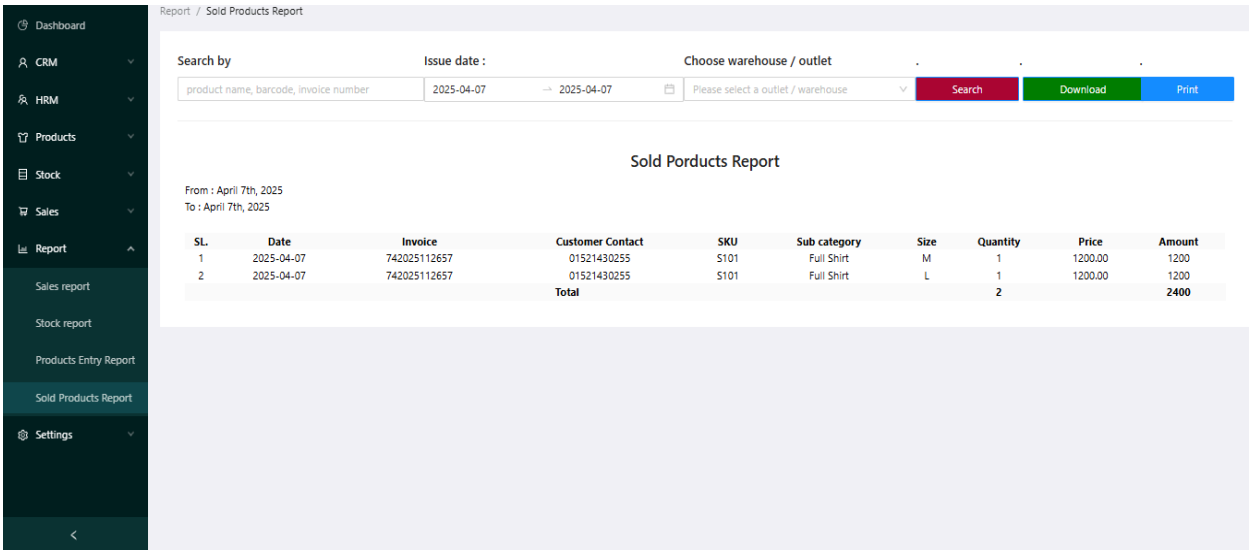


Figure 24. Sold Product Report module

Conclusion

This Inventory and Sales Management Application combines robust API functionality and Frontend with a polished frontend to deliver a comprehensive Inventory and Sales solution. Users can create new products based on variants and sell them with the details of customers. With its scalable design and responsive interface, the application caters to diverse user needs, making it an ideal platform for modern ERP. This project demonstrates the potential of Django Rest Framework and ReactJS in creating efficient, dynamic, and user-centered web applications.

List of used literature

1. Rahmet Bee, "Sales and Inventory Management System," UTPedia, 2019. Discusses the development of a sales and inventory system aimed at replacing manual processes to improve efficiency, focusing on database design, report generation, and point of sale functionality. [Source: http://utpedia.utp.edu.my/13591/1/Rahmet%20Bee_13743.pdf]

2. Patrick Thomas Lucas, "Use of Inventory Management for Large General Item Retailers," Schreyer Honors College, The Pennsylvania State University, 2013. Applies inventory management theory to retail settings, optimizing inventory policies using ABC classification and cost models for holding and shortage costs, with [Source: https://honors.libraries.psu.edu/files/final_submissions/2045]

3. GitLab CI Documentation - https://docs.gitlab.com/ee/ci/

4. Ant Design DocumentationOfficial documentation for Ant Design, the React UI framework used to build elegant and user-friendly interfaces, supporting responsive design and accessibility.[Source: Ant Design official site]

5. PostgreSQL DocumentationAuthoritative source on PostgreSQL features, performance tuning, indexing, and data integrity mechanisms essential for building a robust backend database for enterprise applications.[Source: PostgreSQL official site]

6. ReactJS Official Documentation Provides comprehensive guidelines on building component-based frontend applications with React, including state management techniques and integration with REST APIs.[Source: ReactJS official site]

7. Django Rest Framework Best Practices Guide - 2025," StudyGyaan, 2024Covers key DRF development practices such as API versioning, class-based views, serialization techniques, and structuring projects for maintainability and performance.[Source: StudyGyaan Blog]

8.Escape.tech, "How to Secure Your Django Applications," 2023 Provides security best practices for Django apps, including authentication, authorization, input validation, and protection against common vulnerabilities, which are crucial for backend APIs handling sensitive business data.[Source: Escape.tech Blog]

9,Getting started with Django Rest Framework by Building a Simple Product Inventory Manager" – Techie Diaries Step-by-step tutorial on creating a REST API for a product inventory manager with models, serializers, and views explained in detail.[Techie Diaries Tutorial]"

10.Inventory Management System API" – GitHub Repository by mbuthimungaiA complete inventory management system API built with Django REST Framework, featuring item, stock, transaction, category, and supplier management, containerized with Docker and using PostgreSQL.[GitHub Repo]

11."Developing an Inventory Management System with Django and Bootstrap" – mwass.hashnode.devTutorial on building an inventory system backend with Django and frontend with Bootstrap, useful for understanding integration of UI frameworks with Django.[Hashnode Article]

12"Django DRF Ecommerce Inventory with Elasticsearch and Pytest" – YouTube by Dennis IvyVideo tutorial demonstrating building an ecommerce inventory system with Django REST Framework, Elasticsearch for search, and Pytest for testing.[YouTube Video]

13.Katana MRP Blog (2024).

Inventory Management API Benefits and Process.

This industry article details how inventory management APIs enable integration with ecommerce, POS, and warehouse systems, supporting real-time inventory updates, automated order fulfillment, and demand forecasting-key features for modern Sales and Inventory web applications.[Katana MRP Blog]

14. Rest API [https://www.ibm.com/think/topics/rest-apis]

Appendix A. Backend API routes

```
GET /api/order/
```

```
HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
    "deliverytype": "http://127.0.0.1:8000/api/order/deliverytype/",
    "invoices": "http://127.0.0.1:8000/api/order/onlineorder/",
    "invoicesP": "http://127.0.0.1:8000/api/order/onlineorder/",
    "items": "http://127.0.0.1:8000/api/order/solditems/",
    "items-copy": "http://127.0.0.1:8000/api/order/items-copy/",
    "exchange": "http://127.0.0.1:8000/api/order/onlineorder/",
    "invoicepayments": "http://127.0.0.1:8000/api/order/invoicepayments/",
    "solditems": "http://127.0.0.1:8000/api/order/solditems/",
    "measurement": "http://127.0.0.1:8000/api/order/measurement/",
    "purchase": "http://127.0.0.1:8000/api/order/purchaseP/",
    "purchaseP": "http://127.0.0.1:8000/api/order/purchaseP/",
    "purchaseitem": "http://127.0.0.1:8000/api/order/purchaseitem/",
    "wordrobe": "http://127.0.0.1:8000/api/order/wordrobe/",
    "wordrobeitem": "http://127.0.0.1:8000/api/order/wordrobeitemP/",
    "wordrobeitemP": "http://127.0.0.1:8000/api/order/wordrobeitemP/",
    "Services": "http://127.0.0.1:8000/api/order/ServicesP/",
    "ServicesP": "http://127.0.0.1:8000/api/order/ServicesP/",
    "costings": "http://127.0.0.1:8000/api/order/costings/",
    "cupon": "http://127.0.0.1:8000/api/order/cupon/",
```

Figure 25 - API routes for Order module

```
GET /api/product/
```

```
HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
    "attribute": "http://127.0.0.1:8000/api/wholesale/attributes/",
    "term": "http://127.0.0.1:8000/api/wholesale/attribute-terms/",
    "category": "http://127.0.0.1:8000/api/wholesale/category/",
    "categoryProduct": "http://127.0.0.1:8000/api/wholesale/category/",
    "updatecategory": "http://127.0.0.1:8000/api/wholesale/category/",
    "singpleproduct": "http://127.0.0.1:8000/api/product/productwithalldetails/",
    "productwithalldetails": "http://127.0.0.1:8000/api/product/productwithalldetails/",
    "Details": "http://127.0.0.1:8000/api/product/product/",
    "DetailsUpdate": "http://127.0.0.1:8000/api/product/product/",
    "productentryledger": "http://127.0.0.1:8000/api/product/productentryledger/",
    "DetailsUpdateCustom": "http://127.0.0.1:8000/api/product/product/",
    "product": "http://127.0.0.1:8000/api/product/product/",
    "image": "http://127.0.0.1:8000/api/product/image/",
    "transfer": "http://127.0.0.1:8000/api/product/transferP/",
    "transferP": "http://127.0.0.1:8000/api/product/transferP/",
    "transferitem": "http://127.0.0.1:8000/api/product/floating-transfer-items/",
    "BarcodePrintList": "http://127.0.0.1:8000/api/product/BarcodePrintList/",
    "floating-transfer-items": "http://127.0.0.1:8000/api/product/floating-transfer-items/"
}
```

Figure 26 - API routes for Product module